Networking on linux

1. Getting Info About the Network Interfaces (`ifconfig`, `ip`, `route`)

`ifconfig`, `ip`, and `route` are essential commands for managing network interfaces in Linux. `ifconfig` displays network interface details, while `ip` is a more modern tool for configuring IP addresses and routes. The `route` command shows and modifies the IP routing table, helping manage network traffic paths.

a. **Displaying Interface Information**

- **Using `ifconfig`**
  - `ifconfig`: Display information about enabled interfaces.
  - `ifconfig -a`, `ip address show`: Display information about all interfaces (enabled and disabled).
  - `ifconfig enp0s3`, `ip addr show dev enp0s3`: Display info about a specific interface.
- **Using `ip`**
  - `ip -4 address`: Show only IPv4 info.
  - `ip -6 address`: Show only IPv6 info.
  - `ip link show`: Display L2 info including MAC address.
  - `ip link show dev enp0s3`: Display L2 info for a specific interface.
- **Using `route`**
  - `route`: Display the default gateway.
  - `route -n`: Display numerical addresses for the default gateway.
  - `ip route show`: Display the routing table.
  - `systemd-resolve --status`: Display the DNS servers.

2. Setting the Network Interfaces

a. **Disabling an interface**

- `ifconfig enp0s3 down`
- `ip link set enp0s3 down`

b. **Activating an interface.**

- `ifconfig enp0s3 up`
- `ip link set enp0s3 up`

c. **Checking Interface Status**

- `ifconfig -a`
- `ip link show dev enp0s3`

d. **Setting an IP Address on an Interface**

- `ifconfig enp0s3 192.168.0.222/24 up`
- `ip address del 192.168.0.111/24 dev enp0s3`
- `ip address add 192.168.0.112/24 dev enp0s3`

e. **Setting a Secondary IP Address on a Sub-Interface**

- `ifconfig enp0s3:1 10.0.0.1/24`

f. **Deleting and Setting a New Default Gateway** using `route`

- **Using** `route`
  - `route del default gw 192.168.0.1`
  - `route add default gw 192.168.0.2`
- **Using** `ip`
  - `ip route del default`
  - `ip route add default via 192.168.0.1`

g. **Changing the MAC Address**

- **Using** `ifconfig`
  - `ifconfig enp0s3 down`
  - `ifconfig enp0s3 hw ether 08:00:27:51:05:a1`

- `ifconfig enp0s3 up`

- **Using** `ip`
    - `ip link set dev enp0s3 address 08:00:27:51:05:a3`

---

3. Network Static Configuration Using netplan (Ubuntu)

a. **Stopping and Disabling the Network Manager**

- `sudo systemctl stop NetworkManager`: Stop the Network Manager.
- `sudo systemctl disable NetworkManager`: Disable the Network Manager.
- `sudo systemctl status NetworkManager`: Check the status of the Network Manager.
- `sudo systemctl is-enabled NetworkManager`: Verify if the Network Manager is disabled.

b. **Creating a YAML Configuration File in** `/etc/netplan`

YAML Configuration Example:

```
yamlCopy code
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.0.20/24
      gateway4: "192.168.0.1"
      nameservers:
        addresses:
          - "8.8.8.8"
          - "8.8.4.4"
```

c. **Applying and Checking the New Configuration**

- `sudo netplan apply`: Apply the new Netplan configuration.
- `ifconfig`: Display network interface configuration.
- `route -a`: Display the routing table.

---

4. OpenSSH (Secure Shell)

a. **Usage**

- Secure Remote Management of Servers, Routers, other Networking Devices
- Network File Copy: rsync, scp, sftp, winscp
- Tunneling, SSH Port Forwarding
- **sshd** is the **SSH server (daemon)** and **ssh** or **putty** is the client.

b. **Installation of OpenSSH (client and server)**

- **Ubuntu**: `sudo apt update && sudo apt install openssh-server openssh-client`
- **CentOS**: `sudo dnf install openssh-server openssh-clients`
- `sudo systemctl status ssh`: Checking the status of the SSH service.
- `sudo systemctl [start|restart|stop] ssh`: Start, restart, or stop the SSH service.
- `sudo systemctl [enable|disable] ssh`: Enable or disable SSH auto-start on boot.
- `/etc/ssh/sshd_config`: Server configuration file.
- `/etc/ssh/ssh_config`: Client configuration file.

c. **Controlling the SSHd Daemon**

```
ssh -p 22 username@server_ip      # Connect using default S
ssh -p 22 -l username server_ip  # Connect with a specific
ssh -v -p 22 username@server_ip  # Connect in verbose mode

# Ubuntu
sudo systemctl status ssh       # Check SSH status
```

```
sudo systemctl stop ssh          # Stop SSH service
sudo systemctl restart ssh       # Restart SSH service
sudo systemctl enable ssh        # Enable SSH to start on bo
sudo systemctl is-enabled ssh    # Check if SSH is enabled o

# CentOS
sudo systemctl status sshd        # Check SSH status
sudo systemctl stop sshd          # Stop SSH service
sudo systemctl restart sshd       # Restart SSH service
sudo systemctl enable sshd        # Enable SSH to start on b
sudo systemctl is-enabled sshd    # Check if SSH is enabled
```

d. **Securing and Hardening the SSHd daemon**

- `man sshd_config` : Manpage of sshd_config for detailed configuration options.
- Change the configuration file **(/etc/ssh/sshd_config**) like the following and then restart the server
  1. **Change the Port**: `Port 2278`
  2. **Disable Direct Root Login**: `PermitRootLogin no`
  3. **Limit Users' SSH Access**: `AllowUsers stud u1 u2 john`
  4. **Activate Public Key Authentication**: `PubkeyAuthentication yes`
  5. **Disable Password Authentication**: `PasswordAuthentication no`
  6. **Use Only SSH Protocol Version 2**: `Protocol 2`
  7. **Other configurations:**
     a. `ClientAliveInterval 300`
     b. `ClientAliveCountMax 0`
     c. `MaxAuthTries 2`
     d. `MaxStartUps 3`
     e. `LoginGraceTime 20`

---

5. Copying Files using SCP and RSYNC

`scp` (Secure Copy) and `rsync` are powerful command-line tools used for transferring files between local and remote systems securely. `scp` uses SSH to copy files with encryption, ensuring data security during transfer. It's simple but doesn't handle partial transfers efficiently. `rsync`, on the other hand, is more versatile, offering features like incremental file transfer, bandwidth control, and file synchronization. It only transfers the changed parts of files, making it faster and more efficient for syncing large directories or performing regular backups.

SCP

- `scp a.txt john@80.0.0.1:~`: Copy a local file to a remote destination.
- `scp -P 2288 a.txt john@80.0.0.1:~`: Copy a local file to a remote destination using a custom port.
- `scp -P 2290 john@80.0.0.1:~/a.txt .`: Copy a file from a remote destination to the current directory.
- `scp -P 2290 -r projects/ john@80.0.0.1:~`: Copy a local directory to a remote destination using the `r` option for recursion.

RSYNC

- `sudo rsync -av /etc/ ~/etc-backup/`: Synchronize a directory.
- `sudo rsync -av --delete /etc/ ~/etc-backup/`: Mirror a directory, deleting files from the destination that no longer exist in the source.
- `rsync -av --exclude-from='~/exclude.txt' source_directory/ destination_directory/`: Exclude files listed in the `exclude.txt` file during synchronization.
    - **exclude.txt** example:
        - `.avi`
        - `music/`
        - `abc.mkv`
- `rsync -av --exclude='*.mkv' --exclude='movie1.avi' source_directory/ destination_directory/`: Exclude specific files from synchronization.
- `sudo rsync -av -e ssh /etc/ student@192.168.0.108:~/etc-backup/`: Synchronize a directory over the network using SSH.
- `sudo rsync -av -e 'ssh -p 2267' /etc/ student@192.168.0.108:~/etc-backup/`: Synchronize a directory over the network using SSH with a custom port.

6. WGET - File Downloading Tool

`wget` is a command-line utility used for downloading files from the web. It supports HTTP, HTTPS, and FTP protocols, making it a versatile tool for retrieving content from remote servers. One of its key features is the ability to resume interrupted downloads, ensuring reliable file transfers. Additionally, `wget` can download entire websites for offline viewing, making it a powerful tool for managing downloads and automating data retrieval in scripts.

a. **Installing Wget**

- `apt install wget` : Install wget on Ubuntu.
- `dnf install wget` : Install wget on CentOS.

b. **Downloading Files**

- `wget https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Download a file to the current directory.
- `wget -c https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Resume a previously stopped download.
- `mkdir kali` : Create a directory named **kali**.
- `wget -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Save the file into the **kali** directory.

c. **Advanced Download Options**

- `wget --limit-rate=100k -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Limit download speed to 100kB per second while saving to the **kali** directory.
- `wget -i urls.txt` : Download multiple files from URLs listed in **urls.txt**.

d. **Background Downloads**

- `wget -b -P kali/ https://cdimage.kali.org/kali-2020.2/kali-linux-2020.2-installer-amd64.iso` : Start the download in the background.
- `tail -f wget-log` : Monitor the download log to check its status.

e. **Downloading Websites**

- `wget --mirror --convert-links --adjust-extension --page-requisites --no-parent http://example.org`: Get a full offline copy of a website.
- `wget -mkEpnp http://example.org`: An alternative command to mirror a website for offline viewing.

---

7. NETSTAT and SS

`netstat` is a command-line tool used for displaying network connections, routing tables, interface statistics, and more. It helps in monitoring network activity, troubleshooting issues, and seeing which ports are open or being used by different processes. However, it is gradually being replaced by `ss` due to its faster performance and more detailed output.

`ss` (Socket Statistics) is a more modern and efficient utility than `netstat` for displaying socket-related information. It provides detailed statistics on active connections, showing TCP, UDP, and Unix sockets. It is faster, consumes fewer resources, and offers advanced filtering options to narrow down the network data you need to analyze.

a. **Displaying Open Ports and Connections**

- `sudo netstat -tupan`: Display all open ports and connections.
- `sudo ss -tupan`: Display open ports and connections using **ss**.
- `netstat -tupan | grep :80`: Check if port 80 is open.

8. LSOF (List Open Files)

a. **Listing Open Files**

- `lsof`: List all open files on the system.
- `lsof -u username`: List all files opened by processes of a specific user.
- `lsof -c sshd`: List all files opened by a specific process (e.g., **sshd**).
- `lsof -iTCP -sTCP:LISTEN`: List all open TCP ports.
- `lsof -iTCP -sTCP:LISTEN -nP`: List open TCP ports, showing numeric addresses without resolving hostnames.

---

9. Scanning Hosts and Networks using Nmap

`nmap` (Network Mapper) is a powerful open-source tool used for network discovery and security auditing. It is widely used by network administrators and penetration testers to identify hosts and services on a network, detect open ports, and determine potential vulnerabilities. With various scanning techniques, `nmap` can detect operating systems, identify running services, and create a comprehensive network map, making it an essential tool for network security and troubleshooting. Only scan your own hosts and servers. Scanning networks is your responsibility.

a. **Basic Scans**

- `nmap -sS 192.168.0.1`: Perform a SYN scan (half-open scan, requires root).
- `nmap -sT 192.168.0.1`: Perform a TCP connect scan.

b. **Port Scans**

- `nmap -p- 192.168.0.1`: Scan all ports (0-65535).
- `nmap -p 20,22-100,443,1000-2000 192.168.0.1`: Scan specific ports (20, 22-100, 443, 1000-2000).

c. **Version and OS Detection**

- `nmap -p 22,80 -sV 192.168.0.1`: Detect services and versions.
- `nmap -O 192.168.0.1`: Perform OS detection.
- `nmap -A 192.168.0.1`: Enable OS detection, version detection, script scanning, and traceroute.

d. **Network and Host Scans**

- `nmap -sP 192.168.0.0/24`: Perform a ping scan on an entire network.
- `nmap -Pn 192.168.0.0/24`: Treat all hosts as online and skip host discovery.
- `nmap -sS 192.168.0.0/24 --exclude 192.168.0.10`: Exclude a specific IP from the scan.

e. **Saving Output and Target Lists**

- `nmap -oN output.txt 192.168.0.1`: Save scan results to a file.
- `nmap -p 80 -iL hosts.txt`: Read target IPs from a file and scan port 80.
- `nmap -n -iL hosts.txt -p 80 -oN output.txt`: Scan from a list of hosts, disable reverse DNS, and save output to a file.